Module 2: Sv39 MMU, Page Tables, and TLB Acceleration

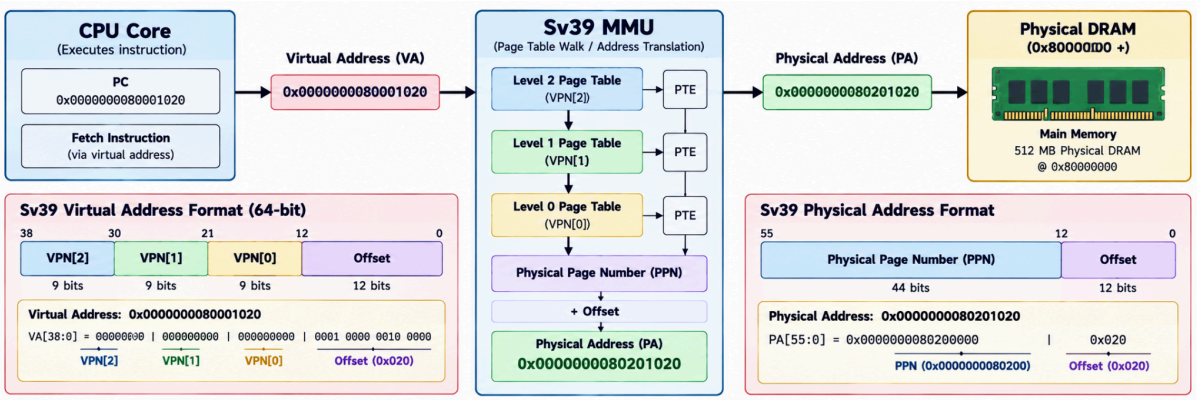
Step 1: Learning Objectives & Prerequisites

- **Learning Objectives:** By the end of this module, you will be able to:
    1. Deconstruct a 39-bit RISC-V Virtual Address into its 9-bit Virtual Page Numbers (**VPN[2]**, **VPN[1]**, **VPN[0]**) and 12-bit **Page Offset**.
    2. Parse the **satp** (Supervisor Address Translation and Protection) CSR bitfields.
    3. Trace a 3-level page table walk from root Physical Page Number (**PPN**) down to a leaf Page Table Entry (**PTE**).
    4. Decode PTE access permission bits (**V**, **R**, **W**, **X**, **U**, **A**, **D**) and explain Copy-on-Write (**COW**) behavior.
    5. Explain TLB hit/miss mechanics and why **SFENCE.VMA** is required when altering page mappings.
- **Prerequisites:** Module 1 (CPU registers, PC execution loop), hexadecimal address arithmetic, and bitwise manipulation.

1. The Core Hardware Problem

An operating system like Linux executes multiple processes simultaneously. If processes directly accessed physical RAM using raw addresses, a buggy or malicious application could corrupt kernel memory or read secrets from another process.

To prevent this, the CPU's **Memory Management Unit (MMU)** sits between the CPU core and the physical memory bus. It translates private **Virtual Addresses (VA)** generated by instructions into shared **Physical Addresses (PA)** in DRAM.



2. Mental Model: A 3-Tier Filing Cabinet

Think of virtual address translation as finding a document in a huge library filing cabinet:

- **satp Register:** Points to the location of the **Main Building (Root Page Table at Level 2)**.
- **VPN[2] :** Tells you which **Room** to enter.
- **VPN[1] :** Tells you which **Cabinet** inside that room to open.
- **VPN[0] :** Tells you which **Folder** inside that cabinet to pull out.
- **Page Offset :** The exact **Page/Line Number** inside the document itself.

3. Bitfield Structures in RISC-V Sv39

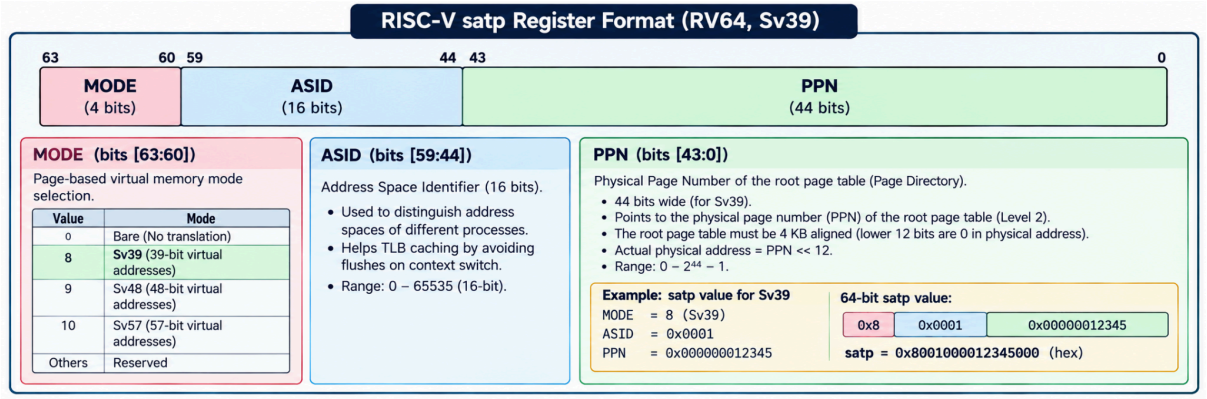
A. Virtual Address (39 bits valid, sign-extended to 64 bits)

RISC-V Sv39 uses a 39-bit virtual address space ($2^{39} = 512GiB$ per process). Bits [63:39] must match bit 38 (canonical address check).

Page Size: $2^{12} = 4096bytes(4KiB)$.
Entries per table: Each page table page is $4KiB$ and contains 512 entries ($512 \times 8bytes/PTE = 4096bytes$). A 9-bit index ranges from 0 to 511.

B. satp CSR Layout (64-bit)

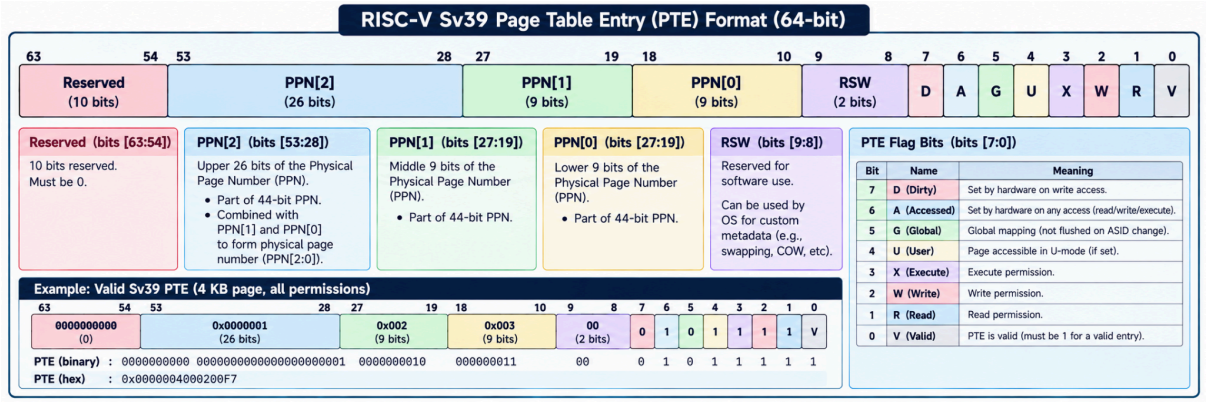
Control register governing supervisor address translation:



- **MODE = 0 (Bare):** Virtual memory disabled; $VA = PA$.
- **MODE = 8 (Sv39):** Page-based 3-level virtual memory enabled.
- **PPN:** Physical Page Number of the Root (Level-2) Page Table. Root physical address = $PPN \times 4096 = PPN \ll 12$.

C. Page Table Entry (PTE) Layout (64-bit)

Each page table entry stores translation mappings and security permissions:



- **Flags (Bits [7:0]):**
  - **V (Valid):** Must be `1` for entry to be active.
  - **R (Read), W (Write), X (Execute):**
    - If `R = 0, W = 0, X = 0`: This entry points to the **next-level page table page**.
    - If any of `R, W, X` is `1`: This entry is a **Leaf PTE** pointing to an actual physical RAM page.
  - **U (User):** Accessible in User-mode (U-mode) if `1`.
  - **A (Accessed):** Set to `1` by hardware/emulator upon read/write/fetch.
  - **D (Dirty):** Set to `1` by hardware/emulator upon a store operation.

Step 3: Source-Code Investigation (`yantra` Implementation)

In emulator, MMU address translation and TLB caching logic follow a structured C++ flow:

```cpp
struct TlbEntry {
    uint64_t vpn;       // Virtual Page Number
    uint64_t ppn;       // Translated Physical Page Number
    uint8_t flags;      // Permission flags (R, W, X, U, etc.)
    bool valid;
};

class Mmu {
private:
    TlbEntry tlb[256];  // Direct-mapped or associative TLB cache

public:
    uint64_t translate(
        uint64_t va,
        AccessType access_type,
        PrivilegeMode mode
    ) {
```

```
    // 1. Bare Mode Check
    if (satp.mode == SATP_MODE_BARE ||
        mode == PRIV_MACHINE) {
        return va;  // Direct physical mapping
    }

    uint64_t vpn = va >> 12;       // Extract Virtual Page Number
    uint64_t offset = va & 0xFFF;  // Extract 12-bit page offset

    // 2. TLB Lookup
    TlbEntry* entry = tlb_lookup(vpn);

    if (entry && entry->valid) {
        check_permissions(
            entry->flags,
            access_type,
            mode
        );

        return (entry->ppn << 12) | offset;
    }

    // 3. TLB Miss -> 3-Level Page Table Walk
    uint64_t pa = page_walk(va, access_type, mode);
    return pa;
}

uint64_t page_walk(
    uint64_t va,
    AccessType access_type,
    PrivilegeMode mode
) {
    uint64_t vpn[3] = {
        (va >> 12) & 0x1FF,  // VPN[0]
        (va >> 21) & 0x1FF,  // VPN[1]
        (va >> 30) & 0x1FF   // VPN[2]
    };

    // Start at Root Page Table (Level 2)
    uint64_t a = satp.ppn << 12;

    for (int i = 2; i >= 0; --i) {
        // Calculate Page Table Entry address
        uint64_t pte_addr = a + (vpn[i] * 8);

        // Read 64-bit Page Table Entry
```

```
            uint64_t pte = bus.read64(pte_addr);

            // Check whether the PTE is valid
            if (!(pte & PTE_V)) {
                raise_page_fault(va, access_type);
            }

            // Check if this is a Leaf Entry (R, W, or X)
            if (pte & (PTE_R | PTE_W | PTE_X)) {
                check_permissions(
                    pte,
                    access_type,
                    mode
                );

                // Extract Physical Page Number
                uint64_t ppn =
                    (pte >> 10) & 0x003FFFFFFFFFFULL;

                // Populate TLB
                tlb_insert(vpn_full, ppn, pte);

                // Construct Physical Address
                return (ppn << 12) | (va & 0xFFF);
            }

            // Not a leaf: Move to the next-level page table
            a = ((pte >> 10) & 0x003FFFFFFFFFFULL) << 12;
        }

        // Reached Level 0 without finding a Leaf Entry
        raise_page_fault(va, access_type);
    }
};
```

Step 4: Concrete Walkthrough (3-Level Page Walk)

Let's trace a virtual memory access for address **0x0000000080001020** (Read Access in U-mode):

Initial Parameters

- `va = 0x0000000080001020`
- `satp.ppn = 0x80200` (Root Table Physical Address = `0x80200000`)
- `VPN[2] = 0x002` (bit bits [38:30])
- `VPN[1] = 0x000` (bits [29:21])
- `VPN[0] = 0x001` (bits [20:12])
- `Offset = 0x020` (bits [11:0])

Walk Execution Steps

Level 2 (Root Lookup):

- Read PTE at address: `Root_PA + (VPN[2] * 8) = 0x80200000 + (2 * 8) = 0x80200010`.
- Loaded `PTE_2 = 0x0002010000000001` (`V=1, R=0, W=0, X=0`).
- **Result:** Non-leaf entry. Points to Level-1 table address: `0x20100 << 12 = 0x80400000`.

Level 1 Lookup:

- Read PTE at address: `0x80400000 + (VPN[1] * 8) = 0x80400000 + 0 = 0x80400000`.
- Loaded `PTE_1 = 0x0002020000000001` (`V=1, R=0, W=0, X=0`).
- **Result:** Non-leaf entry. Points to Level-0 table address: `0x20200 << 12 = 0x80800000`.

Level 0 (Leaf Lookup):

- Read PTE at address: `0x80800000 + (VPN[0] * 8) = 0x80800000 + (1 * 8) = 0x80800008`.
- Loaded `PTE_0 = 0x000205000000001F` (`V=1, R=1, W=1, X=1, U=1, A=1, D=1`).
- **Result:** Leaf entry! `PPN = 0x20500`.
- Physical Page Base Address: `0x20500 << 12 = 0x81400000`.

Final Physical Address Calculation:

- `PA = Physical Page Base + Offset = 0x81400000 | 0x020 = 0x81401020`.

Step 5: Edge Cases & Real-World Considerations

1. **Invalid Page Faults:**
- If `PTE_V == 0`, or an attempt is made to **Write** to a page where `PTE_W == 0`, the MMU triggers a Page Fault Exception (`Load Page Fault`, `Store/AMO Page Fault`, or `Instruction Page Fault`).
2. **Copy-on-Write (COW):**
- Operating systems like Linux optimize process creation (`fork()`) by marking shared pages as `Read-Only` (`W=0`). When a child process attempts to write to the page, a `Store Page Fault` triggers. The kernel copies the 4 KiB page, sets `W=1` for the new copy, and updates the PTE.
3. **Superpages (Gigapages & Megapages):**
- If a leaf PTE (`R/W/X != 0`) is encountered at **Level 2**, it maps a **1 GiB Gigapage**.
- If a leaf PTE is encountered at **Level 1**, it maps a **2 MiB Megapage**.
- Superpages reduce page table walking overhead and improve TLB efficiency for kernel memory.
4. **SFENCE.VMA Instruction:**
- Whenever the OS alters page table entries or switches `satp`, old translations in the emulator's TLB become stale. The CPU executes `SFENCE.VMA`, causing the emulator to clear cached TLB entries.

Step 6: System Connectivity

- **Upstream Dependency:** Receives raw virtual addresses generated during instruction fetch (`PC`) and memory execution (`LOAD`/`STORE`) from the **CPU Core (Module 1)**.
- **Downstream Dependency:** Converts valid VAs into physical addresses, routing memory requests to the physical **DRAM Subsystem (0x80000000)** or **MMIO Peripheral Bus (UART, PLIC, CLINT, VirtIO)**.

Step 7: Summary & Review

Key Takeaways:

1. **Sv39** translates 39-bit virtual addresses to 56-bit physical addresses using 3-level page tables ($4KiB$ pages).
2. The **satp** CSR holds the root page table physical address (`PPN`) and enables MMU translation (`MODE = 8`).
3. An entry with `R=0, W=0, X=0` points to a sub-table; any set `R/W/X` bit designates a leaf page.
4. A **TLB** caches recent virtual-to-physical translations to skip expensive 3-memory-lookup page walks.

5. **SFENCE.VMA** flushes stale TLB mappings when page tables change.